

Templating

Templating in Flask is the process of creating dynamic web pages by combining HTML with data from the Flask application.

Flask uses the **Jinja2 template engine** to generate HTML pages dynamically.

In standard web development, Python code handles business logic, database queries, and routing, while HTML handles the presentation layer. Templating bridges the gap between them, allowing backend data to be rendered dynamically inside HTML documents without writing HTML strings directly inside Python code.

```
mirror_mod = modifier_ob.  
# Add mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly  
-- OPERATOR CLASSES ----  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
context):  
context.active_object is not
```

Need for Templating

- **Separation of Concerns:** Keeps Python logic completely separate from HTML design, making the code cleaner, easier to debug, and maintainable by different people (e.g., developers vs. UI designers). Separates the presentation layer from application logic.
- **Dynamic Data Rendering:** Allows loops, conditions, and variables directly inside HTML using Jinja2 syntax, enabling pages to change based on user input or database records.
Makes web pages dynamic and reusable.
- **Code Reusability:** Features like template inheritance allow developers to create a single base layout (header, footer, navigation bar) and reuse it across multiple pages.
Reduces code duplication.
- Simplifies the maintenance of web applications.
- **Security (Auto-Escaping):** Jinja2 automatically escapes HTML by default, protecting applications against Cross-Site Scripting (XSS) attacks.





How Templating Works in Flask

- The user sends a request to the Flask application.
- The Flask view function processes the request.
- Data is passed from the view function to the template.
- The template engine generates the final HTML page.
- The generated page is sent back to the user's browser.

How Flask Organizes and Renders Templates

Flask expects all HTML template files to be stored in a specific folder named **templates** located in the root directory of your project.

To render a template, Flask provides a function called `render_template()`.

This function searches the templates directory, processes the Jinja2 syntax, and sends the final HTML output to the client's browser.



Advantages of Django

- Built-in admin interface.
- Integrated ORM and authentication system.
- Strong security features.
- Suitable for enterprise-level applications.

Advantages of Flask

- Lightweight and easy to learn.
- Highly flexible and customizable.
- Ideal for APIs and small web applications.
- Minimal setup and configuration.

- Choose **Django** when developing **large, complex, and database-driven applications** that require many built-in features.
- Choose **Flask** when developing **small, lightweight, and highly customizable applications** or REST APIs.

Django follows the principle "batteries included," while Flask follows the principle "minimal core with maximum flexibility."

Feature	Django	Flask
Type of Framework	Full-stack web framework	Micro web framework
Complexity	More complex due to many built-in features	Simple and lightweight
Development Speed	Faster for large applications because many tools are built in	Faster for small applications but requires additional packages for larger projects
Architecture	Follows the MVT (Model-View-Template) architecture	Does not enforce a specific architecture
Database Support	Includes built-in ORM for database operations	Uses external libraries such as SQLAlchemy for ORM support
Admin Interface	Provides a built-in admin panel	No built-in admin panel
Flexibility	Less flexible because of predefined structure	Highly flexible and customizable
Learning Curve	Steeper learning curve	Easier for beginners to learn

Project Size	Suitable for medium and large-scale applications	Suitable for small and medium-scale applications
Security Features	Provides built-in security features against common attacks	Security features need to be implemented using extensions or custom code
URL Routing	Uses a centralized URL configuration system	Uses decorators such as <code>@app.route()</code> for routing
Template Engine	Uses Django Template Language (DTL)	Uses Jinja2 template engine
Community Support	Large and mature community support	Large and active community support

Web Security Considerations in Python Frameworks

Python web frameworks such as Django and Flask provide several security features to protect web applications from cyber threats.

Python frameworks provide strong security mechanisms to protect web applications. While Django offers many security features by default, Flask provides flexibility through extensions. Proper use of these features helps developers build secure and reliable web applications.

1. Authentication

- Verifies the identity of users through login systems.
- Supports username/password authentication and multi-factor authentication.

2. Authorization

- Controls access to resources based on user roles and permissions.
- Ensures users can access only authorized data and functions.

3. Protection Against SQL Injection

- Frameworks use ORM (Object Relational Mapping) to prevent direct SQL query manipulation.
- User inputs are automatically escaped in most database operations.

4. Protection Against Cross-Site Scripting (XSS)

- Templates automatically escape harmful scripts before displaying user data.
- Prevents malicious JavaScript code injection.

5. Protection Against Cross-Site Request Forgery (CSRF)

- Uses CSRF tokens to verify legitimate requests.
- Prevents unauthorized actions performed by attackers.

6. Secure Session Management

- Session data is stored securely.
- Session cookies can be protected using secure and HTTP-only flags.

7. Password Hashing

- Passwords are stored in encrypted hash format rather than plain text.
- Makes it difficult for attackers to retrieve actual passwords.

8. Input Validation

- Validates user inputs before processing.
- Prevents invalid or malicious data from entering the system.

9. HTTPS Support

- Supports secure communication using HTTPS and SSL/TLS encryption.
- Protects sensitive information during transmission.

10. Regular Updates

- Developers should update frameworks and libraries regularly to fix vulnerabilities.

Security Features in Django and Flask

Security Feature	Django	Flask
Built-in Authentication	Yes	Available through extensions
CSRF Protection	Built-in	Available through extensions
SQL Injection Protection	Built-in ORM	Supported through ORM libraries
XSS Protection	Automatic escaping in templates	Automatic escaping in Jinja2 templates
Session Management	Built-in	Supported through extensions
Password Hashing	Built-in	Supported through libraries

Advantages of Security Features in Python Frameworks

- Reduces development effort.
- Protects applications from common attacks.
- Improves data confidentiality and integrity.
- Enhances user trust and application reliability.

Comparison of web security considerations of Django and Flask

Security Vector	Django Approach	Flask Approach
Core Security Philosophy	Secure-by-default: Comprehensive built-in middleware and protections handle common web threats out of the box.	Customizable flexibility: Minimalist core leaves configuration and security extension management to the developer.
Cross-Site Scripting (XSS)	Automatically escapes all variables rendered inside templates by default, requiring explicit bypasses (<code>`</code>	<code>safe`</code>).
CSRF Protection	Active globally via <code>CsrfViewMiddleware</code> on state-changing requests, validated using template tokens (<code>{% csrf_token %}</code>).	Omitted from core routing; requires third-party packages like Flask-WTF to inject and check tokens.
SQL Injection (SQLi)	Handled natively by the Django ORM, which automatically parameterizes queries and escapes inputs.	Relies on libraries like SQLAlchemy , where ORM queries are parameterized by default unless raw string concatenation is used.
Authentication & Password Hashing	Fully integrated <code>contrib.auth</code> system featuring robust password hashing (PBKDF2/Argon2) and session control.	Requires modular extensions like Flask-Login or Flask-JWT-Extended , alongside <code>Werkzeug.security</code> hashing functions.
Security Headers & Cookies	Built-in <code>SecurityMiddleware</code> automatically sets HSTS, content-type options, and secure session flags.	Requires manual header configuration or wrapper extensions like Flask-Talisman for CSP and HTTPS enforcement.
	Leaving <code>DEBUG = True</code> exposes detailed traceback logs and	Leaving <code>DEBUG = True</code> runs the interactive